

Le système Unix

Vincent Nicomette, Matthieu Herrb, Thierry Monteil

INSA Toulouse

2ième IMACS-MIC

Version 2020

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

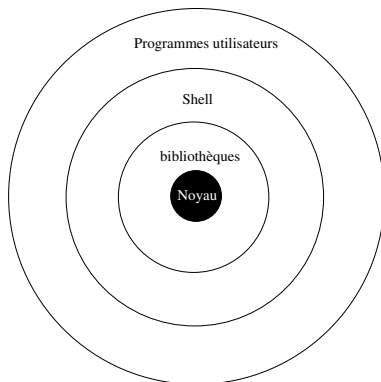
Historique

- ▶ Unix est né aux laboratoires Bell (Filiale d'ATT)
- ▶ Développé en assembleur à partir de 1969 par Ken Thompson et Dennis Ritchie
- ▶ En 1973, Unix est réécrit à 90% en langage C (créé pour l'occasion par Brian Kernighan et Dennis Ritchie)
- ▶ A l'origine, Unix est un système d'exploitation pour stations de travail et serveurs ; il est également devenu, avec l'émergence des Unix "libres", un système d'exploitation de machine personnelle
- ▶ Fournisseurs : Digital Equipment, Hewlett Packard, IBM, Silicon Graphics, Sun Microsystems, etc.
- ▶ Percée des Unix "libres" : Gnu/Linux, OpenBSD, etc.
- ▶ Concurrent : Windows de Microsoft

Vue générale (1/2)

Système organisé en couches successives

- ▶ au coeur le noyau (kernel)
- ▶ des bibliothèques permettant la programmation du système
- ▶ le Shell (interpréteur de commandes)
- ▶ des programmes utilisateurs



Vue générale (2/2)

- ▶ Caractéristiques principales :
 - ▶ système multi-tâches en temps partagé
 - ▶ système multi-utilisateurs
 - ▶ système intégré aux réseaux
 - ▶ Langages de commande :
 - ▶ le Bourne Shell (`sh`)
 - ▶ le Korn Shell (`ksh`)
 - ▶ le C Shell (`csh`)
 - ▶ le TC Shell (`tcsh`)
 - ▶ le Bourne-Again Shell (`bash`)
- bash et tcsh sont les plus utilisés.**
- ▶ Outils :
 - ▶ manipulation de texte
 - ▶ développement de logiciels
 - ▶ communication
 - ▶ documentation
 - ▶ bureautique

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

Identification des utilisateurs

- ▶ Chaque utilisateur est uniquement identifié dans le système par un nom de *login* auquel sont associées les informations suivantes :
 - ▶ un mot de passe
 - ▶ un identificateur numérique (*uid*)
 - ▶ un identificateur numérique de groupe(*gid*)
 - ▶ un commentaire (informations diverses sur l'individu)
 - ▶ un répertoire de travail
 - ▶ un langage de commandes
- ▶ Toutes ces informations (sauf le mot de passe) sont stockées dans un fichier local (/etc/passwd) ou une base de données centralisée disponible à travers le réseau telle qu'une base LDAP)
- ▶ Le mot de passe **chiffré** est stocké dans un autre fichier (/etc/shadow)
- ▶ Un utilisateur particulier (**root**) gère le système (administrateur)

Connexion des utilisateurs

- ▶ L'accès au système peut se faire depuis :
 - ▶ un terminal graphique (clavier, écran, souris)
 - ▶ par le réseau
- ▶ Déroulement de la connexion :
 - ▶ le programme (*login*) demande à l'utilisateur d'entrer son nom de login ainsi que son mot de passe
 - ▶ le programme (*login*) vérifie la validité du mot de passe et lance le shell associé à l'utilisateur ; l'utilisateur est automatiquement placé dans le répertoire de travail qui lui est associé
- ▶ Exemple :

```
login: eleve
Password: <password invisible>
Last login: Tue Sep 7 15:22:12 on console
machine[~]
```

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

**Le système de
fichiers**

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

Structure (1/4)

Le système Unix

Vincent
Nicomette,
Matthieu Herrb,
Thierry Monteil

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

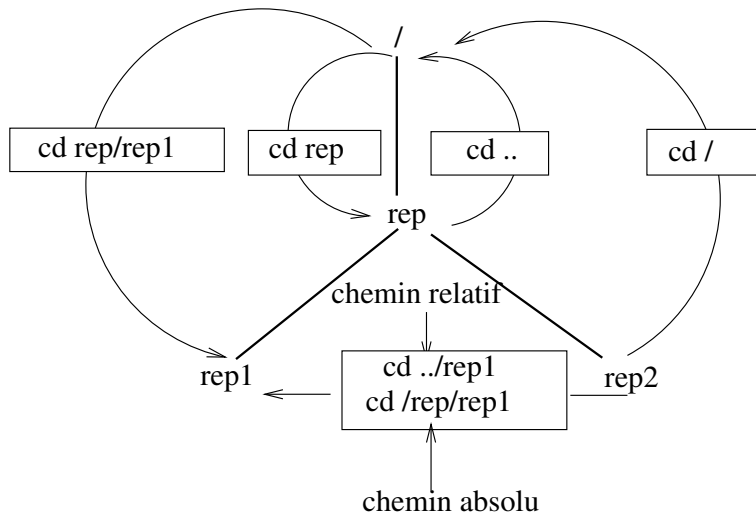
Divers

- ▶ Notion de fichier très importante sous Unix : “Tout est fichier”
- ▶ Quatre types de fichiers :
 - ▶ ordinaire : contient des données
 - ▶ répertoire : contient d'autres fichiers ou répertoires
 - ▶ lien symbolique : pointe vers un autre fichier ou répertoire
 - ▶ spécial : permet l'accès à un périphérique
- ▶ Un fichier est représenté par une structure (l-node) qui stocke les informations sur un fichier : taille, droits d'accès, dates de création, de modification ...

Structure (2/4)

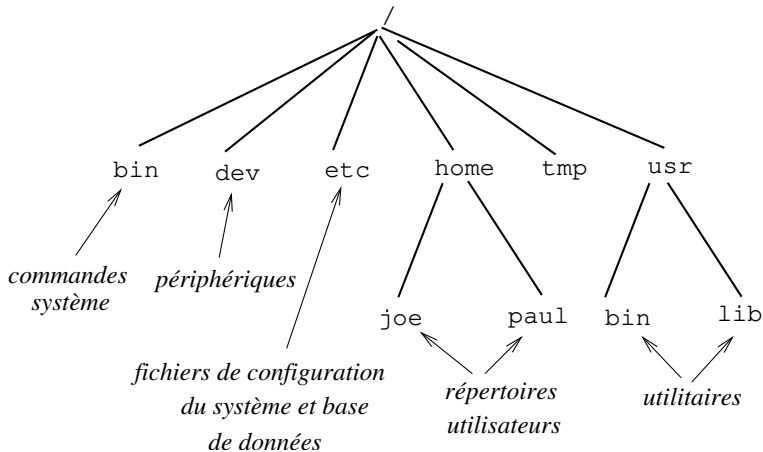
- ▶ **Structure arborescente** du système de fichiers
- ▶ Le répertoire racine est noté /
- ▶ Le caractère / désigne également le séparateur de répertoires
- ▶ Un *chemin absolu* représente de manière unique le chemin d'accès à un fichier en partant du répertoire racine
Exemple de chemin absolu : /home/prof/interro/sujet
- ▶ Un *chemin relatif* à un répertoire donné représente le chemin d'accès en partant de ce répertoire
- ▶ Le *répertoire courant* désigne le répertoire dans lequel l'utilisateur se trouve à l'instant courant
- ▶ Caractéristique des noms de fichiers :
 - ▶ tous les caractères sont valides sauf "/" et le code ASCII 0
 - ▶ **les majuscules sont distinguées des minuscules**
- ▶ Le répertoire courant est noté .
- ▶ Le répertoire père (immédiatement situé au-dessus dans l'arborescence) est noté ..

Structure (3/4)



Structure (4/4)

► Les répertoires usuels

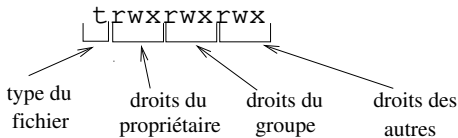


Droits d'accès (1/2)

- ▶ Pour un fichier ou répertoire, trois catégories d'utilisateurs :
 - ▶ le propriétaire : u
 - ▶ le groupe : g
 - ▶ les autres : o
- ▶ Pour un fichier ou répertoire, trois types de droits :
 - ▶ lecture : r (pour un répertoire, signifie lire le contenu du répertoire)
 - ▶ écriture : w (pour un répertoire, signifie pouvoir créer ou détruire un fichier dans le répertoire)
 - ▶ exécution : x (pour un répertoire, signifie pouvoir traverser ce répertoire)
- ▶ Le type d'un fichier est noté :
 - ▶ fichier ordinaire : -
 - ▶ répertoire : d
 - ▶ lien symbolique : l
 - ▶ fichier spécial : c ou b

Droits d'accès (2/2)

- Les droits d'accès à un fichier ou un répertoire sont exprimés par une chaîne de 10 caractères :



- L'absence de droit est notée : - (Ex : `-r-xr-xr-x`)
- Le super utilisateur (root) a **tous** les droits sur les fichiers de la machine qu'il administre, quels que soient les droits indiqués par la chaîne de caractères
- Représentation octale des droits :
Ex : `rw-----` : 600 `rwxr-x---` : 750

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

Introduction

- ▶ Le shell est un interpréteur de commandes dont le fonctionnement est le suivant :
 - ▶ il affiche un message (le *prompt*), attend la frappe d'une ligne, analyse cette ligne et exécute la commande correspondante
 - ▶ pour exécuter une commande, il crée un *processus* (cf. *slides sur les processus*)
- ▶ Certains shells (`ksh`, `csch`, `tcsh`, `bash`) disposent de possibilités interactives plus étendues (historique, complétion, etc)
- ▶ Le shell dispose de variables, de structures de contrôle et de structures itératives qui en font un langage de commande
- ▶ Un fichier de configuration est associé au shell :
 - ▶ `csch` : `.cschrc`
 - ▶ `tcsh` : `.tcshrc`
 - ▶ `bash` : `.bashrc`
- ▶ Un shell par défaut est associé à chaque utilisateur (son shell de login)
- ▶ **Ce cours se focalise sur le shell `bash`**

Commandes internes/externes

- ▶ `bash` interprète des commandes *internes* (*built-in command*) ou *externes*
 - ▶ commandes *internes* : commandes directement interprétées par le shell (Ex : `cd`)
 - ▶ commandes *externes* : commandes auxquelles sont associées un programme exécutable (souvent dans le répertoire `/usr/bin` ou `bin`) => le shell cherche ce programme et l'exécute
- ▶ La structure générale d'une commande est :
`commande arguments`
`arguments` est une liste de paramètres passés à la commande ; trois formes sont possibles :
 - ▶ option booléenne : `-option`
Ex : `ls -l`
 - ▶ option avec valeur : `-option valeur`
Ex : `lp -d mon_imprimante`
 - ▶ chemin d'accès à un fichier
Ex : `cat /home/prof/corrige`

Les métacaractères

- Des caractères qui ont une signification particulière

Caractère	Signification
/	répertoire racine, séparateur de répertoires
.	répertoire courant
..	répertoire père
~	home-directory
' et "	délimiteurs de chaînes de caractères
cmd1 ; cmd2	exécution de cmd1 puis de cmd2
cmd1 cmd2	exécution de cmd2 si cmd1 échoue
cmd1 && cmd2	exécution de cmd2 si cmd1 réussit
> >> <	<i>cf. slides sur les pipes</i>
&	<i>cf. slides sur les processus</i>
' \$ (())	<i>cf. slides sur la programmation</i>
\	neutralise le métacaractère suivant

Les métacaractères de substitution

Caractère	Signification
*	n'importe quelle suite de caractères, y compris vide
?	n'importe quel caractère
[...]	un caractère de la liste. Ex : [A-Z0-9]

► Exemples :

Si le répertoire courant contient :

fich1.bin, fich1.txt, fich2.txt, fich10.txt,
fichier.txt, readme, zzz

alors :

fich1* => fich1.bin, fich1.txt, fich10.txt

fich*.txt => fich1.txt, fich2.txt, fich10.txt,
fichier.txt

fich[0-9]*.txt => fich1.txt, fich2.txt, fich10.tx

??? => zzz

► Une *expression régulière* est une expression utilisant des métacaractères

Les variables

- ▶ `bash` dispose d'un mécanisme de création de variables
 - ▶ elles peuvent être *standards* (leur nom est imposé) ou créées par l'utilisateur
 - ▶ elles peuvent être *locales* (connues uniquement du shell qui les créé) ou *exportables* (connues de tous les processus descendants du processus qui les a créées, cf. *slides sur les processus*)
 - ▶ Les variables locales sont aussi appelées variables *privées* du shell
 - ▶ Les variables exportables sont aussi appelées variables *d'environnement*
- ▶ Les variables ne sont pas typées
- ▶ Si une variable s'appelle `var`, on accède à son contenu par `$var`

Les variables privées

- ▶ Peuvent être standards ou définies par l'utilisateur
- ▶ Par défaut, elles ne sont pas transmises lors de la création de processus (*cf. slides sur les processus*)
- ▶ Quelques exemples de variables standards
 - ▶ IFS : le séparateur de champs
 - ▶ PS1 : le prompt
- ▶ La commande `set` permet de visualiser les variables privées du shell

Les variables d'environnement (1/2)

- ▶ Peuvent être standards ou définies par l'utilisateur
- ▶ Par défaut, elles sont transmises lors de la création de processus (*cf. slides sur les processus*)
- ▶ Définition d'une variable :

```
VARIABLE=valeur; export VARIABLE
```
- ▶ Consultation de la valeur d'une variable :

```
printenv VARIABLE ou echo $VARIABLE
```
- ▶ Un fichier de configuration `.bash_profile` contient des commandes exécutées au début de chaque connexion utilisateur, il peut être utilisé pour positionner des variables d'environnement

Les variables d'environnement (2/2)

- ▶ Quelques variables d'environnement standards importantes :
HOME : désigne le répertoire de connexion de l'utilisateur
PATH : spécifie les chemins de recherche des commandes
→ cette variable est **fondamentale**
- ▶ Manipulation de la variable PATH
 - ▶ initialisation de PATH :

```
bash$ PATH=/usr/bin:/usr/sbin:/bin
bash$ export PATH
```
 - ▶ concaténation de PATH :

```
bash$ PATH=/home/moi/bin:$PATH
```
- ▶ La commande `printenv` ou `env` permet de visualiser les variables d'environnement

Manipulation des variables (1/5)

- ▶ Il est possible de définir des variables non typées (pas de déclaration de variables)
- ▶ Un nom de variable peut comporter des lettres, des chiffres et le caractère _
- ▶ Initialisation d'une variable s'effectue en même temps que l'affectation (grâce à l'opérateur =)

Ex : `bash$ x=7; c=Paris; x=$y`

Pas d'espace avant et après = !!

- ▶ Le délimiteur guillemets admet la substitution contrairement aux apostrophes

```
bash$ c=Paris
bash$ echo "La ville de $c"
La ville de Paris
bash$ echo 'La ville de $c'
La ville de $c
```

- ▶ On peut concaténer des variables

```
bash$ a=Haute; b=Garonne; c=$a$b
```

Manipulation des variables (2/5)

Le système Unix

Vincent
Nicomette,
Matthieu Herrb,
Thierry Monteil

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

► Utilisation des accolades { }

```
bash$ var=truc  
bash$ var1=machin  
bash$ echo $var1  
machin  
bash$ echo ${var}1  
truc1
```

Manipulation des variables (3/5)

- Manipulation de chaînes de caractères (sous-chaînes, substitutions)

```
bash$ chaine=123456789abcd
bash$ echo ${#chaine}      # la longueur
13
bash$ echo ${chaine:5}
6789abcd
bash$ echo ${chaine:3:8}
456789ab
bash$ a=BB40
bash$ b=${a/BB/AA}
bash$ echo $b
AA40
bash$ a=tatatiti
bash$ b=${a//ta/to}
bash$ echo $b
tototiti
```

Manipulation des variables (4/5)

► Variables en lecture seule

```
bash$ declare -r var=truc
bash$ echo $var
truc
bash$ var=machin
bash: var: readonly variable
```

► Possibilité d'utiliser des tableaux :

```
bash$ tab=(lundi mardi mercredi) # ou
bash$ tab[0]=lundi; tab[1]=mardi; tab[2]=mercredi
bash$ echo ${tab[0]}
lundi
bash$ echo ${tab[*]}           # tout le tableau
lundi mardi mercredi
bash$ echo ${#tab[*]}          # le nb d'elements
3
```

Manipulation des variables (5/5)

- ▶ Exportation d'une variable par la commande `export`
`export var` (`var` est alors connue des *processus* descendants et devient une variable d'environnement)
- ▶ Saisie clavier : utilisation de la commande `read`
Saisie de la variable `v` :

```
bash$ read v
toto
bash$ echo $v
toto
```

Saisie d'une ligne :

```
bash$ read
il fait beau
bash$ echo $REPLY
il fait beau
```

→ la ligne saisie est dans la chaîne `REPLY`

Scripts bash : introduction

- ▶ Un “script” est un ensemble de commandes internes ou externes rassemblées dans un fichier
- ▶ Pour lancer l'exécution du script `prog.sh`, deux possibilités :
 1. `bash$ bash prog.sh [paramètres]`
 2. `bash$ ./prog.sh [paramètres]` (après avoir positionné le droit d'exécution sur le fichier)
- ▶ La première ligne du script doit indiquer le type de shell qui doit être utilisé pour l'exécuter : `#!/chemin-du-shell`
Ex : `#!/bin/bash`
Attention : si cette ligne est absente du fichier, le bourne shell est automatiquement utilisé si on lance le script en utilisant `./prog.sh`
- ▶ Le caractère `#` en début de ligne indique une ligne de commentaires
- ▶ Possibilité d'utiliser des variables, des opérations arithmétiques des structures de contrôle et des fonctions

Scripts bash : opérations arithmétiques (1/2)

- ▶ Une variable peut être considérée comme entière en fonction des opérations qu'on lui applique
- ▶ Utilisation de l'opérateur `expr` pour les opérations arithmétiques
 - ▶ `resultat='expr arg1 operateur arg2'` (espaces entre les lexèmes)
 - ▶ l'expression `'cmd'` ou `$(cmd)` exprime le résultat de la commande `cmd` (sa sortie)

```
bash$ n=2
bash$ n='expr $n + 1'
bash$ echo $n
3
bash$ n=$(expr $n + 1)
bash$ echo $n
4
bash$ a=3; b=4
bash$ c='expr $a \* $b' # neutralisation de *
bash$ echo $c
12
```


Scripts bash : opérations arithmétiques (2/2)

Le système Unix

Vincent
Nicomette,
Matthieu Herrb,
Thierry Monteil

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

► Utilisation de let

```
bash$ i=3; j=4  
bash$ let resultat=i+j  
bash$ echo $resultat  
7
```

► Utilisation de doubles parenthèses

```
bash$ x=3; y=4  
bash$ z=$((x + y))  
bash$ echo $z  
7
```

Scripts bash : variables prédéfinies (1/2)

- ▶ Un script shell peut avoir des paramètres qui sont référencés dans le script par :
\$1, \$2, ..., \$9, \${10}, ...
- ▶ \$0 désigne le nom du script lui-même
- ▶ \$# représente le nombre de paramètres passés au script (\$0 non compris)
- ▶ \$* représente la liste des paramètres passés au script
- ▶ Ex : soit le script shell suivant :

```
#!/bin/sh
echo "execution du programme : $0"
echo "nombre de parametres : $#"
```

```
echo "voici le 1er parametre : $1"
```

Son exécution donne :

```
bash$ ./script.sh on est les champions
execution du programme : ./script.sh
nombre de parametres : 4
voici le 1er parametre : on
```

Scripts bash : variables prédéfinies (2/2)

- ▶ \$\$ désigne le numéro du processus créé pour exécuter le script
- ▶ \$? désigne le code de retour de la dernière commande exécutée
 - ▶ si ce code est 0, la commande s'est exécutée correctement
 - ▶ si ce code est différent de 0, la commande a échoué
 - ▶ Ex :

```
bash$ cd /  
bash$ echo $?  
0  
bash$ cd /existepas  
bash$ echo $?  
1
```

Scripts bash : fonctions

- ▶ On peut définir des fonctions dont les paramètres sont manipulés dans la fonction par les variables \$1, \$2, ...
→ pas de déclaration de paramètres
- ▶ Le passage des paramètres se fait comme le passage d'arguments à une commande : `Fonc param1 param2 ...`
- ▶ Utilisation de `return` pour renvoyer le retour d'une fonction ; ce retour est récupéré par l'appelant dans la variable \$?
- ▶ Exemple de script shell

```
Mul() {                # fonction Mul
    let r=$1*$2         # 2 parametres
    return $r           # retourne le resultat
}

Mul 10 12              # appel de la fonction
r=$?                  # recuperation du resultat
echo $r               # r vaut 120
```

Scripts bash : structures de contrôle (1/4)

► La structure de contrôle de condition

```
if condition1
then
    commandes1
elif condition2
then
    commandes2
else
    commandes3
fi
```

- les conditions sont souvent exprimées à l'aide de la commande `test` (*cf. slide suivant*)

Scripts bash : structures de contrôle (2/4)

- La commande `test` fait l'évaluation des expressions booléennes

- Exemples à deux paramètres :

`test $i -eq 3` : égalité pour les valeurs entières

`test $i -ne 3` : non égalité pour les valeurs entières

`test $i -gt $j` : strict. supérieur pour les valeurs entières

`test $i = $j` : égalité pour les chaînes de caractères

`test $i != $j` : non égalité pour les chaînes de caractères

- Exemples à 1 paramètre :

`test -f file` : renvoie vrai si file est un fichier

`test -d file` : renvoie vrai si file est un dossier

`test -r file` : renvoie vrai si file a le droit r positionné

- Notation équivalente : utilisation des doubles crochets `[[ ]]`

`[[ $i -eq 3 ]]` : égalité pour les valeurs entières

`[[ -f ]]` file : renvoie vrai si file est un fichier

Scripts bash : structures de contrôle (3/4)

- La commande test (suite)
- Exemples de script shell :

```
if test $var -eq 1
then
    echo "$var vaut 1"
fi

if [[ -d $var ]]
then
    echo "$var est un dossier"
fi
```

- Possibilité d'utiliser les doubles parenthèses et les opérateurs standards du langage C : ==, !=, <, <=, >=, !

```
if ((i < j))
then
    echo "plus petit "
fi
```

Scripts bash : structures de contrôle (4/4)

- La structure d'aiguillage à choix multiple

```
case $var in
  valeur1)
    commandes1
    ;;
  valeur2)
    commandes2
    ;;
  ...
  *)
    commandes-def
    ;;
esac
```

- valeur1, valeur2 sont des chaînes de caractères, utilisant éventuellement des métacaractères du shell
- le caractère * signifie n'importe quelle chaîne de caractères

Scripts bash : structures itératives (1/3)

► La boucle for

```
for var in liste
do
    commandes
done
```

- liste est une liste de chaîne de caractères, utilisant éventuellement les métacaractères du shell
- Exemple de script shell :

```
for fichier in *.c
do
    chmod go-r $fichier
done
```

Scripts bash : structures itératives (2/3)

Le système Unix

Vincent
Nicomette,
Matthieu Herrb,
Thierry Monteil

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

- La boucle for (suite)
 - Boucle de 1 à 10, plusieurs possibilités :

```
for i in 1 2 3 4 5 6 7 8 9 10
for i in {1..10}
for ((i=0; i<10; i++))
for i in `seq 1 10`
```

Scripts bash : structures itératives (3/3)

► La boucle while

```
while condition
do
    commandes
done
```

boucle infinie : `while true` ou `while :`

► La boucle until

```
until condition
do
    commandes
done
```

=> exécute commandes jusqu'à ce que condition soit vraie

L'historique - Les alias

- ▶ La mémorisation des commandes
 - ▶ la commande `history` : permet d'obtenir un historique des commandes (chaque commande possède un numéro)
 - ▶ mémorisation de `n` commandes : `HISTSIZE=n`
 - ▶ `!n` : réexécute la commande numéro `n`
 - ▶ `!xxx` : réexécute la dernière commande de l'historique commençant par `xxx`
- ▶ Les alias : permettent de créer des “raccourcis” de commandes complexes
 - ▶ `alias` : visualise la liste des alias
 - ▶ `alias ll="ls -l"` crée l'alias `ll` qui est un raccourci vers la commande `ls -l`

Les raccourcis clavier

- Utilisation de combinaisons de touches facilitant la saisie de commandes :
 - Ctrl-P : commande précédente (flèche haut)
 - Ctrl-N : commande suivante (flèche bas)
 - Ctrl-B : curseur vers la gauche (flèche gauche)
 - Ctrl-F : curseur vers la droite (flèche droite)
 - Ctrl-A : aller au début de la ligne
 - Ctrl-E : aller à la fin de la ligne
 - Ctrl-T : transpose les 2 derniers caractères
 - Ctrl-U : efface la ligne en cours
 - Ctrl-L : efface l'écran
 - Ctrl-D : détruit le caractère sous le curseur / liste les complétions possibles / provoque la deconnexion du shell
 - TAB : complète le nom du fichier ou du répertoire en cours (complétion automatique)

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

**Quelques
commandes**

Redirections et
pipes

Les processus

Divers

Les commandes : manipulation de fichiers

ls [-la...] [fich_ou_rep]

=> Les crochets signifient facultatif!!!!!!

- ▶ liste le contenu d'un répertoire ou donne les infos sur un fichier
- ▶ l'option -a liste en plus les fichiers cachés (commençant par un .)
- ▶ l'option -l affiche des informations sur chaque fichier
- ▶ Ex :

```
machine [UNIX] ls -l
-rw-rw-r-- 1 riviere develop 44032 Sep 13 15:13 unix.fm
-rw-rw-r-- 1 riviere develop 43008 Sep 13 15:04 unix.backup.fm
```

↑ ↑ ↑ ↑ ↑ ↑

droits propriétaire groupe taille date de nom du fichier
d'accès

modification

pwd (commande interne)

- ▶ pour *print working directory*, affiche le chemin absolu du répertoire courant

Les commandes : manipulation de fichiers

`cd [repertoire]`(commande interne)

- ▶ pour *change directory*, permet de se déplacer dans l'arborescence

`mkdir repertoire`

- ▶ crée un répertoire dont le nom est passé en argument de la commande (chaque nom peut être un chemin absolu ou relatif)

`rmdir repertoire`

- ▶ efface un répertoire, **le répertoire doit être vide !!**

`rm [-ri...] fic_ou_rep`

- ▶ sans option, efface le fichier correspondant
- ▶ `-r` permet d'effacement récursif d'un répertoire
- ▶ `-i` provoque la demande de confirmation : très bonne option

Les commandes : manipulation de fichiers

cat fichier

- ▶ affiche le contenu du fichier à l'écran

more fichier

- ▶ affiche le contenu du fichier à l'écran page par page

head [-n] fichier

- ▶ affiche les *n* premières lignes du fichier

tail [-n] fichier

- ▶ affiche les *n* dernières lignes du fichier

mv fic_ou_rep1 fich_ou_rep2

- ▶ renomme et/ou déplace un fichier ou un répertoire

mv fich /tmp : déplace le fichier *fich* dans le répertoire */tmp*

mv rep /tmp/rep2 : déplace le répertoire *rep* dans le
répertoire */tmp* et le renomme *rep2*

Les commandes : manipulation de fichiers

cp [-R...] fic_ou_rep1 fic_ou_rep2

- copie un fichier ou un répertoire

cp fich1 fich2 : fich2 est une copie de fich1 dans le répertoire courant

cp fich1 rep : fich1 est copié dans le répertoire rep

cp -R rep1 rep2 : si rep2 existe, copie le répertoire rep1 dans rep2; si rep2 n'existe pas, copie rep1 dans le répertoire courant, la copie s'appelle rep2

cmp/diff fic_ou_rep1 fic_ou_rep2

- permet de comparer le contenu de 2 fichiers ou répertoires (diff pour les fichiers texte)

file fichier

- permet de déterminer le type d'un fichier (shell script, données, fichier texte, fichier exécutable, ...)

Les commandes : manipulation de fichiers

wc [-l] [-w] [-c] fichier

- ▶ pour *word count*, permet de compter le nombre de lignes (-l), de mots (-w) ou de caractères (-c) d'un fichier

du [-sh...] fic_ou_rep

- ▶ affiche le nombre de blocs occupés sur le disque dur par un répertoire ou un fichier
- ▶ l'option -s indique pour un répertoire la somme totale seulement et non un affichage détaillé par fichier
- ▶ l'option -h permet un affichage "human-readable"

sort [options] fichier

- ▶ permet de trier les lignes d'un fichier (par défaut, ordre alphabétique)

type commande (commande interne)

- ▶ indique si la commande est une commande interne ou externe

Les commandes : recherche d'information

find repertoire [options expression]

- fait la recherche récursive descendante à partir du répertoire indiqué, des fichiers qui vérifient l'expression spécifiée

find . -name core -size +0 : cherche à partir du répertoire courant, les fichiers de nom core dont la taille est supérieure à 0 et affiche leur chemin

grep expression fichier

- recherche la chaîne de caractères expression dans le fichier spécifié

grep oula *.txt : cherche parmi tous les fichiers dont l'extension est txt ceux qui contiennent la chaîne oula

Les commandes : gérer les droits d'accès

`chmod [-R...] droits fic_ou_rep`

- ▶ change les droits d'accès des fichiers spécifiées; -R permet de parcourir récursivement tous les sous-répertoires
- ▶ `droits` est de la forme : `u|g|o +|- r|w|x` ou de la forme octale.

`chmod g+w fich` : ajoute le droit d'écriture à tous les membres du groupe du fichier

`chmod 750 fich` : positionne **tous** les droits d'accès sur le fichier

- ▶ **seul le propriétaire d'un fichier (et root) peut changer les droits d'accès de ce fichier**

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

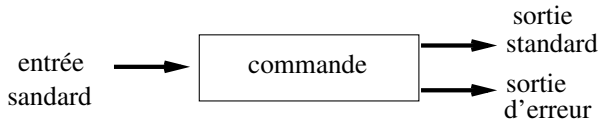
**Redirections et
pipes**

Les processus

Divers

Redirections (1/3)

- ▶ Chaque commande a :
 - ▶ une entrée standard (par défaut le clavier)
 - ▶ une sortie standard (par défaut l'écran)
 - ▶ une sortie d'erreur (par défaut l'écran)
- ▶ Unix est conçu à l'origine de telle façon que les commandes soient très simples et très nombreuses : chaque commande lit ses entrées sur l'entrée standard et écrit ses résultats sur la sortie standard (néanmoins, chaque commande est libre d'**utiliser ou pas** cette entrée et ces sorties)



- ▶ Les opérations complexes se réalisent par combinaison de commandes simples
- ▶ La redirection est un mécanisme simple qui permet de rediriger l'entrée standard ou la sortie standard ou la sortie d'erreur d'une commande

Redirections (2/3)

► Redirection de l'entrée standard

```
commande < fichier
```

la commande lit ses entrées dans `fichier` et non plus au clavier

► Redirection de la sortie standard

```
commande > fichier
```

la commande écrit ses sorties dans `fichier` et non plus à l'écran

```
commande >> fichier
```

la commande concatène ses sorties dans `fichier` et non plus à l'écran

► Redirection de la sortie d'erreur

```
commande 2> fichier
```

la sortie d'erreur est redirigée dans `fichier`

Redirections (3/3)

► Redirection des sorties standard et erreur

```
commande > fichier1 2>fichier2
```

la sortie standard est redirigée dans fichier1, la sortie d'erreur est redirigée dans fichier2

```
commande > fichier 2>&1
```

la sortie standard et la sortie d'erreur sont redirigées dans le même fichier

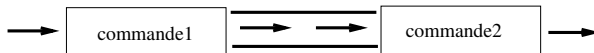
Les pipes (1/2)

- Un pipe est un tube de communication entre deux commandes de telle façon que la sortie standard d'une commande soit redirigée vers l'entrée standard de l'autre

```
commande1 | commande2
```

qui est équivalent à :

```
commande1 > fich; commande2 < fich; rm fich
```



- Exemple :

```
ls -l /tmp | grep riviere
```

la recherche de la chaîne `riviere` se fait dans le fichier généré par la sortie de la commande `ls -l /tmp`

- **Ne fonctionne que si** `commande1` écrit ses données sur la sortie standard et si `commande2` lit ses données sur l'entrée standard

Les pipes (2/2)

- ▶ Un filtre est une commande élémentaire qui, par défaut, prend ses données sur l'entrée standard et fournit ses résultats sur la sortie standard
- ▶ Le filtre est la commande de base utilisée dans les pipes
- ▶ Exemple de filtres :
 - ▶ `sort` : trie les lignes
 - ▶ `uniq` : supprimer les lignes identiques
 - ▶ `head, tail, grep`
- ▶ Exemple de plusieurs filtres :

```
cat fichier | sort | uniq | more
```

trie les lignes d'un fichier par ordre alphabétique puis supprime les lignes identiques et affiche le résultat page par page

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

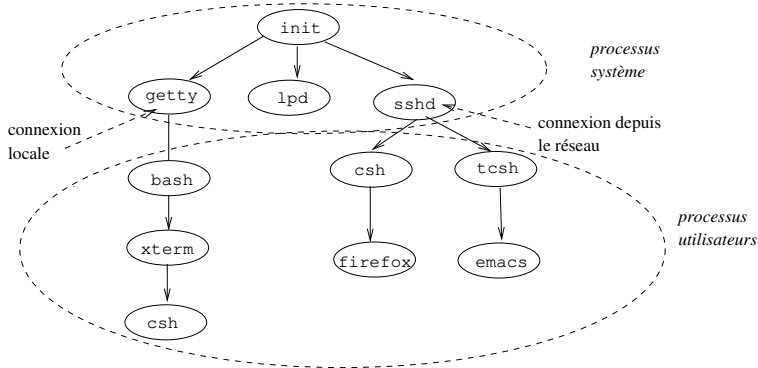
Les processus

Divers

Définition

- ▶ Un processus est l'exécution d'un programme exécutable à un instant donné : deux exécutions successives du même programme sont deux processus différents !!
- ▶ Un processus est identifié de façon unique par un numéro : le **PID**
- ▶ Un processus est créé par un autre processus (mécanisme de filiation)
- ▶ Un processus appartient à l'utilisateur qui l'a lancé
- ▶ Les processus de root sont des processus système
- ▶ Plusieurs états d'un processus
 - ▶ utilise la CPU (*running*)
 - ▶ prêt à utiliser la CPU (*idle*)
 - ▶ suspendu en attente de ressources
 - ▶ transféré en mémoire de masse
 - ▶ mort avant que le processus père ait exécuté la primitive wait (em zombie)
 - ▶ ...

Hierarchie des processus



La commande ps

- ▶ La commande qui permet de lister les processus du système est :

ps

- ▶ Différentes options de la commande ps (options System V) :
 - ▶ -e : visualise tous les processus du système
 - ▶ -f : affichage détaillé
 - ▶ -u uid : affichage des processus de l'utilisateur d'identificateur uid

```
machine[~] ps -ef | more
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	345	1	0	Aug 18	?	0:01	/usr/sbin/sshd
nicomett	1380	1365	0	13:41:30	pts/1	0:01	-tcsh

propriétaire *identificateur du processus (PID)* *identificateur du père du processus* *heure de lancement* *terminal de rattachement* *nom du programme exécutable*

- ▶ **ps auxx** est aussi couramment utilisé

Lancement d'un processus

- ▶ Deux types de lancement de processus
 - ▶ *foreground* : le processus “garde la main” pendant son exécution ; il utilise le clavier et empêche donc son utilisation dans le terminal de lancement
 - ▶ *background* : le processus “rend la main” une fois lancé ; il continue sans faire d'entrée sur le terminal de lancement
- ▶ Lancement d'un processus en foreground :
`commande` (ou `./commande` si la variable PATH n'inclut pas le répertoire courant)
- ▶ Lancement d'un processus en background :
`commande &`

Manipulation des processus

- Visualisation des processus en background : `jobs`
cette commande affiche un **identifiant local** de chaque processus ; **cet identifiant n'est valable que dans le terminal de lancement du processus !**

```
bash$ jobs  
[1] + Running sleep 120  
[2] + Running sleep 10
```

=> 1 et 2 sont des identifiants locaux

- Terminaison d'un processus en background :
`kill %id_local` (`id_local` affiché par la commande `jobs`)
- Passage de l'état background à l'état foreground :
`fg %id_local`
- Passage de l'état foreground à l'état background (2 étapes) :
`Ctrl-Z` : suspend le processus
`bg [%id_local]` : passage en mode background

Les variables d'environnement

- ▶ A chaque processus est associé un ensemble de variables
- ▶ Ces variables sont en général définies au moment de la connexion au système
- ▶ Exemple de variables : PS1, HOME, PATH, ...
- ▶ Lorsqu'un processus crée un autre processus, seulement une partie de ces variables sont transmises au processus fils : **les variables d'environnement** (*cf. slides sur les variables du shell*)
- ▶ La commande `env` ou `printenv` permet de visualiser ces variables

Les signaux

- ▶ Les *signaux* sont un mécanisme d'interruptions logiciel
- ▶ Deux façons d'envoyer un signal à un processus :
 - ▶ raccourci clavier :
 - `Ctrl-C` : envoie le signal SIGINT (interruption)
 - `Ctrl-Z` : envoie le signal SIGTSTP (suspend le processus ! **NE LE TUE PAS!!!**)
 - `Ctrl-\` : envoie le signal SIGQUIT (fin)

Cette méthode n'est utilisable que depuis le terminal de lancement du processus

- ▶ la commande `kill` :

```
kill [-signal] PID
```

le signal par défaut : SIGTERM (-15)

le signal d'urgence : SIGKILL (-9)

Exemples :

```
kill -9 1246 ou kill -KILL 2146
```

- ▶ Possibilité de masquer ou de dérouter un signal grâce à la commande `trap` :

```
trap 'commande' signal
```

 : exécute commande lorsqu'on reçoit le signal signal

Généralités

Les utilisateurs

Le système de fichiers

Le shell

Quelques commandes

Redirections et pipes

Les processus

Divers

Généralités

Les utilisateurs

Le système de
fichiers

Le shell

Quelques
commandes

Redirections et
pipes

Les processus

Divers

La documentation

- ▶ Unix dispose de toute la documentation de référence en ligne
- ▶ La documentation concerne les commandes usuelles mais aussi les appels système et les fonctions de la bibliothèque standard
- ▶ La commande `man` :
 - `man ls` : donne la documentation de la commande `ls`
- ▶ Si un nom correspond à la fois à une commande et une fonction de bibliothèque par exemple, utiliser l'option `-a`
 - `man -a read` : donne toutes les pages du man intitulée `read`

L'éditeur vi

- ▶ vi est l'éditeur historique d'Unix, il est présent sur tous les Unix
- ▶ Ne nécessite pas d'environnement graphique
- ▶ Versions évoluées : vim et gvim
- ▶ Deux modes : commande et insertion
- ▶ Mode commande
 - ▶ h, j, k, l : touches de déplacement
 - ▶ i : passe en mode insertion
 - ▶ x : efface le caractère sous le curseur
 - ▶ dd : détruit la ligne courante
 - ▶ u : Undo
 - ▶ :w<RET> : sauvegarde le fichier courant
 - ▶ :q<RET> : quitte
 - ▶ :q!<RET> : quitte sans sauver
- ▶ Mode insertion
 - ▶ Permet de taper du texte dans le fichier
 - ▶ <ESC> : passe en mode commande

Compression et archivage (1/2)

- ▶ Les utilitaires `gzip` et `bzip2`
 - ▶ permettent de compresser des fichiers
 - ▶ `gzip fichier` / `bzip2 fichier`
 - ▶ le fichier compressé est `fichier.gz` / `fichier.bz2`
 - ▶ `gunzip/bunzip2` pour décompresser
- ▶ L'utilitaire `tar`
 - ▶ permet de sauvegarder une arborescence dans un seul fichier ou un périphérique de sauvegarde
 - ▶ Options :
 - `c` : créer l'archive
 - `f` : utiliser un fichier pour archiver (et non un périphérique)
 - `v` : mode *verbose*
 - `t` : visualiser le contenu de l'archive
 - `x` : extraire le contenu de l'archive

Compression et archivage (2/2)

- Exemples d'utilisation de tar (suite)

`tar cvf archive.tar rep` (création d'archive `archive.tar`
du répertoire `rep`)

`tar tvf archive.tar rep` (visualisation du contenu de
l'archive)

`tar xvf archive.tar rep` (extraction du contenu de
l'archive)

- Utilisation combinée de tar et gzip/bzip2 pour créer des archives compressées
- Certaines versions de tar permettent de faire la compression en même temps que l'archivage (option `z`)

Quelques commandes réseau (1/2)

► `ssh [-l utilisateur] machine`

- Permet d'obtenir un shell sur une machine distante, sur laquelle on dispose d'un compte utilisateur
- toutes les communications sont chiffrées

```
bash$ ssh raymond@machine.truc.fr
raymond@machine.truc.fr's password: <password>
machine[~] ls
...
machine[~] exit
logout
```

Quelques commandes réseau (2/2)

► `sftp [utilisateur@] machine`

- permet d'effectuer des transferts de fichiers vers une machine distante et inversement
- toutes les communications sont chiffrées
- authentification, puis utilisation des commandes : `pwd`, `ls`, `mget`, `put`, `mput`

```
bash$ sftp raymond@machine.truc.fr
raymond@machine.truc.fr's password: <password>
sftp> cd /tmp
sftp> mget *.txt
mget toto.txt ? (y/n)
...
```